A Tale of Trust and Accuracy: Base vs. Instruct LLMs in RAG Systems

Florin Cuconasu[1]*, Giovanni Trappolini[1], Nicola Tonellotto[2], Fabrizio Silvestri[1]
[1]Sapienza University of Rome, [2]University of Pisa

arXiv:2406.14972v1 [cs.CL] 21 Jun 2024

Abstract

Retrieval Augmented Generation (RAG) represents a significant advancement in artificial intelligence combining a retrieval phase with a generative phase, with the latter typically being powered by large language models (LLMs). The current common practices in RAG involve using "instructed" LLMs, which are fine-tuned with supervised training to enhance their ability to follow instructions and are aligned with human preferences using state-of-the-art techniques. Contrary to popular belief, our study demonstrates that base models outperform their instructed counterparts in RAG tasks by 20% on average under our experimental settings. This finding challenges the prevailing assumptions about the superiority of instructed LLMs in RAG applications. Further investigations reveal a more nuanced situation, questioning fundamental aspects of RAG and suggesting the need for broader discussions on the topic; or, as Fromm would have it, "Seldom is a glance at the statistics enough to understand the meaning of the figures". [1] [2]

1 Introduction

Retrieval Augmented Generation (RAG) (Lewis et al., 2020) is an innovative approach that enhances the capabilities of Large Language Models (LLMs) by integrating retrieval mechanisms into the generative process. At its core, RAG operates by retrieving relevant information from a vast corpus of data and then generating coherent and contextually enriched responses based on this retrieved information. This dual process not only improves the accuracy and relevance of the generated content but also addresses some of the inherent limitations of standalone generative models, such as hallucinations (Huang et al., 2023) and context drift (Wang et al., 2022). The significance of RAG in natural language processing and artificial intelligence cannot be overstated. As the demand for more sophisticated and context-aware AI systems grows, the ability to generate information that is both accurate and contextually relevant becomes crucial (Gao et al., 2024). RAG achieves this by leveraging the vast amount of information available, ensuring that the outputs of the models are informed by up-to-date and contextually appropriate data. This has profound implications for various applications, including conversational AI, information retrieval, and automated content generation (Shuster et al., 2021; Wang et al., 2024). Furthermore, RAG represents a paradigm shift in how we think about and utilize LLMs. Instead of relying solely on the generative power of these models, RAG harnesses the complementary strengths of retrieval systems. This synergy enables the creation of AI systems that are not only more knowledgeable but also more reliable and versatile in their applications (Izacard and Grave, 2021; Zhu et al., 2024).

LLMs are the key component in RAG systems. They are initially pre-trained on the task of next token prediction (Radford et al., 2018), where the LLM learns to predict the next word in a sequence based on the context provided by the preceding words. This extensive pre-training phase involves processing vast amounts of text data, enabling the model to acquire a broad understanding of language, syntax, semantics, and general knowledge. We call this the "base" version. Following this pre-training phase, LLMs typically undergo two stages of refinement to enhance their performance and usability, whose output we call the "instruct". The first stage is that of supervised instruction fine-tuning (SFT), where the goal is to teach the model to follow instructions passed through the prompt. This is carried out by supervised fine-tuning, where the model is trained on a curated dataset incorporating instructions, with the specific goal of improving

*Correspondence to cuconasu@diag.uniroma1.it

[1]Translated from I cosiddetti Sani.

[2]The code and data will be made available at github.com/florin-git/Base-vs-Instruct-LLMs-in-RAG-Systems

<table>
<tr><th>Llama 2 7B</th><th>Llama 2 7B-Chat + Template</th></tr>
<tr><td>You are given a question and you MUST respond by EXTRACTING the answer from one of the provided documents. If none of the documents contain the answer, respond with NO-RES.</td><td>[INST] «SYS»
You are given a question and you MUST respond by EXTRACTING the answer from one of the provided documents. If none of the documents contain the answer, respond with NO-RES.
«/SYS»</td></tr>
<tr><td colspan="2">Documents:
Document [1](Title: Batman Returns) the Penguin. We didn't really officially cast it, but for a short nasty little guy, it's a short list. I ended up writing the character for Danny DeVito. Burgess Meredith (who portrayed the Penguin in the 1960s TV series "Batman") was cast for a little cameo as Tucker Cobblepot...
Document [2](Title: Batman: Mystery of the Batwoman) This is the only time in the DC animated universe that Paul Williams did not voice the Penguin...
Document [3](Title: The Penguin's a Jinx) The Penguin goes to Wayne Manor and returns the actress. He then uses his gas-umbrella to knock out anyone inside the statues...

Question: Who played the part of 'The Penguin' in the TV series 'Batman'?</td></tr>
<tr><td>Answer: Burgess Meredith</td><td>Answer: [/INST] Based on the provided documents, the answer is Danny DeVito</td></tr>
</table>

Figure 1: **Base vs. Instruct + Template under Task Instruction I on TriviaQA**. The figure presents a comparison between the responses generated by two versions of the Llama 2 7B model: the base version and the instruct + template version. Each version is tasked with answering the same question based on the provided documents. The base model correctly identifies the answer as "Burgess Meredith", whereas the instruct + template version incorrectly attributes the answer to "Danny DeVito". *Italic* text denotes the template.

its ability to follow the specified instructions (Taori et al., 2023). The second stage is alignment with human preferences, often referred to as reinforcement learning from human feedback (RLHF) (Ouyang et al., 2022) or similar methods (Rafailov et al., 2023; Hong et al., 2024; Rahman and Xue, 2022). During this phase, the model's outputs are adjusted to better align with human preferences and values. This usually involves iterative processes where human evaluators provide feedback on the model's responses, and the model is further fine-tuned to produce outputs that are not only accurate but also contextually appropriate and aligned with human expectations. In practice, the "instruct" versions of LLMs are the go-to models widely used for RAG tasks (Liu et al., 2023; LangChain, 2023; DSPy, 2023). Moreover, these instruct models often come with a "template", specific usage guidelines that provide a structured approach to utilizing the system effectively.

In this paper, we conduct a principled evaluation of instruct models and their accompanying templates against their base versions in a RAG setting. Surprisingly, our results reveal that base models, without the additional instruct-specific fine-tuning, outperform the instruct models on the task of RAG in our experimental setting. This finding challenges the prevailing assumption that instruct models are inherently superior for such tasks. Upon further investigation, we uncover that the situation is more nuanced, with various factors contributing to the observed performance differences. Our study stimulates a broader discussion on RAG, its methodologies, and evaluation procedures employed to advance the state of the art in this field.

In summary, our contributions are:

- Performance Evaluation: We conduct a principled evaluation comparing instruct models and their templates against base models in the context of RAG, revealing that base models outperform instruct models.
- Nuanced Insights: Through detailed analysis, we uncover the complexities and nuances that influence the performance of RAG systems.
- Pathway for Future Research: Our findings challenge existing assumptions and stimulate further discussion on RAG's state of the art, helping the development of more effective and reliable systems.

2 Background

In this section, we will explore the steps involved in training a large language model, including pre-training, instruction fine-tuning, and alignment. We will then discuss the fundamentals of RAG.

2.1 LLM Training

Here, we illustrate the training processes of LLMs, which consist of at least three main steps: pre-training, which is done on the task of next token prediction, instruction fine-tuning, and preference alignment.

Pre-training. Pre-training for large language models (Radford et al., 2018) involves an extensive unsupervised learning phase, where the model is exposed to a large corpus of text data to learn the underlying statistical properties of natural language. This process employs the next token prediction task, where the model is conditioned on a sequence of tokens $w_{1:i-1}$ and trained to predict the subsequent token w_i, i.e., $p(y) = \prod_i^n p_\theta(w_i|w_{1:i-1})$. By iteratively processing vast and diverse textual datasets, the model learns linguistic patterns, including syntactic structures, semantic relationships, and contextual dependencies. This process delivers what is commonly called a "base" model. This first model is usually improved with further specialized training, as we will see now.

Instruction Fine-Tuning. Instruction fine-tuning is the part of the training process of large language models aimed at enhancing their ability to follow specific directives and perform specialized tasks (Taori et al., 2023). This phase involves further training of the base model on curated datasets that include explicit instructions paired with corresponding responses. The objective is to teach the model how to interpret and execute various types of commands or queries effectively. During instruction fine-tuning, the model is exposed to a wide range of examples that illustrate how to respond to different types of prompts, from answering questions to generating summaries and more complex task-oriented interactions. The primary purpose of instruction fine-tuning is to improve the model's usability and performance in real-world applications. By aligning the model's behavior with the specific needs and expectations of users, instruction fine-tuning ensures that the model can handle diverse and nuanced tasks more effectively, enhancing its overall functionality and practical utility. This process might involve using an instruction template, where specific patterns and structures for providing instructions and expected responses are standardized to facilitate consistent and effective learning. It is worth noticing that this is still achieved with traditional supervised learning, unlike the next phase we are considering.

Aligning to Human Preferences. The final step, which might include multiple sub-steps, is aligning LLMs to human preferences. Aligning AI systems ensures that these systems can effectively and ethically interact with humans, providing outputs that are useful, respectful, and culturally sensitive. Human preferences are not easily integrated into a differentiable loss function, necessitating specialized techniques. The most popular are Direct Preference Optimization (DPO) (Xu et al., 2024) and Reinforcement Learning from Human Feedback (RLHF) (Ouyang et al., 2022). These techniques solve the following optimization problem:

$$J_r(\pi_\theta) = \mathbb{E}_{\mathbf{x},\mathbf{y}}\left[r(\mathbf{x},\mathbf{y}) - \beta \log \frac{p_\theta(\mathbf{y} \mid \mathbf{x})}{p_{ref}(\mathbf{y} \mid \mathbf{x})}\right]$$

The LLM $p_\theta(\mathbf{y} \mid \mathbf{x})$ is here considered as a policy that following an instruction x generates a text response y. The reward function $r(\mathbf{x}, \mathbf{y})$ reflects human preferences. It takes a prompt $\mathbf{x}$ and the corresponding response $\mathbf{y}$ as input and outputs a scalar value. The reference model p_{ref} is used to regularize the LLM model p_θ with Kullback–Leibler divergence. The hyper-parameter β is a constant to control the degree of regularization. RLHF solves this problem in a two-step procedure; first, it learns a reward model, and then it uses this learned reward model to optimize the equation above. DPO instead skips the first step and directly solves a less general problem by optimizing the policy p_θ over preference data. Despite the importance of aligning large language models with human preferences, the inner workings of these alignment techniques remain largely unclear. Recently, (Hong et al., 2024) showed that applying RLHF can decrease the expected reward of a model but still improve its performance. Currently, it remains unclear which technique is superior, with experimental evidence showing that it depends on the particular application (like DPO being bad at coding (Xu et al., 2024)). There is a significant effort in the scientific community to understand and improve these methods, aiming to develop more transparent and reliable alignment processes. In the remainder of

this paper, we will try to measure its impact on RAG.

2.2 RAG

RAG (Lewis et al., 2020) is a hybrid approach that integrates information retrieval mechanisms with generative language models to enhance the quality and relevance of generated content. In RAG, a retrieval component first identifies and retrieves relevant documents or data from a large corpus based on a given query or context. This retrieved information is then used as supplementary input for the generative model, which synthesizes a response or generates new text informed by the retrieved data. Specifically, given a corpus of documents D and a query q, a retriever is employed to retrieve the set of documents D_k that are more relevant to the query. In the dense setting, like the one we employ, this is achieved through a neural bi-encoder that independently encodes queries and documents in a learned latent vector space where a similarity score, like the cosine similarity, is used to extract the most similar documents, formally: $sim(q, d_i) \propto \overrightarrow{q} \cdot \overrightarrow{d_i}$, where d_i is the i-th document, while $\overrightarrow{q}$ and $\overrightarrow{d_i}$ are the embedding of the query and i-th document, respectively. The top-k documents are then passed to a generative model, like an LLM. The LLM takes an input, once again, the query q and the set of most relevant documents D_k. The problem is then formulated as generating the answer to the query, conditioning on the query, and marginalizing over the retrieved documents:

$$P_{\text{RAG}}(y|q) \approx \prod_{i}^{N} \sum_{d \in \mathcal{D}_k} p_\eta(d|q) p_\theta(y_i|q, d, y_{1:i-1}),$$

where $p_\eta(d \mid q)$ is the retrieval component that provides a (truncated) probability distribution for the top-scoring documents, and $p_\theta(y_i \mid q, d, y_{1:i-1})$ is a probability distribution parameterized by θ that generates the current token based on the previously generated tokens, the query, and the retrieved document.

3 Experimental Methodology

In this paper, we aim to achieve multiple objectives. Firstly, we seek to determine whether base models outperform their instruction versions in the context of RAG. Secondly, we investigate the underlying factors affecting RAG models' performance and the impact of the additional training techniques (i.e., SFT and Alignment) on these systems. To this end, we set up a series of rigorous experiments to methodically evaluate and compare the performance and behavior of base models versus their instruct-tuned counterparts, providing a comprehensive understanding of their respective advantages and limitations in RAG tasks.

3.1 Task Instructions

We perform our experimenting with two task instructions. Task instruction I (Figure 1) requires the LLM to "extract" the answer from one of the provided documents, reflecting the extractive nature of the QA datasets. Additionally, the models are tasked to respond with *NO-RES* if the answer is not present in the retrieved documents, testing their *negative rejection* capabilities (Chen et al., 2024). To check if the model is really using the provided context and it is aware of it, we also employ another task instruction. Task instruction II builds on the first instruction by asking the model to provide evidence from the given context to support its response with a *Proof*. This instruction includes a one-shot example to demonstrate the expected format of the model's answer, as visualized in Figure 5.

The full prompt is composed of the task instruction, to which the retrieved documents and the query are added. Documents are ordered by ascending similarity score to position the high-similarity documents nearest to the query, according to the insights of Liu et al. (2023).

3.2 Instruct Templates

When fine-tuning LLMs to create their instruct versions, specific prompt templates are used during training. These templates are designed to clearly distinguish between model responses or task instructions and user inputs. This distinction is marked through the use of special tokens. For instance, Llama 3 utilizes <|*begin_of_text*|>, and Mistral uses *[INST]* to specify the beginning of instructions. Despite the use of these templates for instruct LLMs, their effects on model performance, when removed from the typical conversational setting and applied to a rag setting, remain understudied. Our study explores this gap by evaluating the performance of instruct models with and without their standard chat templates.

3.3 Datasets and Models

In our experiments, we use two open-domain question-answering datasets: the open version of Natural Questions (NQ-open) (Kwiatkowski et al., 2019; Lee et al., 2019) and TriviaQA-unfiltered (Joshi et al., 2017). For each query, the Contriever (Izacard et al., 2021) is used to retrieve the most similar documents from the English Wikipedia corpus. The retriever's performance is discussed in Section A.2. For generation, we employ several LLMs in both their base and instruct/chat versions: **(a)** Llama 2 7B and Llama 2 7B-Chat; **(b)** Llama 3 8B and Llama 3 8B-Instruct; **(c)** Falcon 7B and Falcon 7B-Instruct; **(d)** Mistral 7B and Mistral 7B-Instruct; Models are quantized at 4-bit. For clarity, we will refer to the Llama 2 chat version collectively as "instruct". All models utilize greedy decoding, and the maximum response length is tailored to the requirements of each dataset. Under Task Instruction I, the response limit is set to 15 tokens for the NQ dataset, which demands short responses of no more than 5 tokens, and up to 50 tokens for the TriviaQA dataset to accommodate potentially longer answers. For Task Instruction II, which requires the proof, the maximum response length is increased to 200 tokens.

3.4 Evaluation

Accuracy. Accuracy is the main metric adopted to evaluate the models' responses. In particular, it is checked whether one of the ground truth answers of the dataset is contained in the generated response after applying a normalization process. This normalization involves the removal of punctuation and articles to ensure that the answer is not unfairly penalized by minor discrepancies in formatting. This type of evaluation may have flaws since it can incorrectly mark a correct response as incorrect if the ground truth answer is not fully contained in the generated response, even after normalization. Nevertheless, given that the employed datasets usually require short answers (e.g., answers in NQ-open are at most 5 tokens long) that can be extracted from the provided documents, adopting accuracy may be generally considered a suitable metric.

Negative Rejection. As shown in Figure 1, LLMs are tasked with responding with *NO-RES* when the provided documents do not contain the necessary knowledge to answer the query. This approach helps assess the models' abilities to understand the task instructions and correctly refuse to answer when the information is not present, thereby reducing the occurrence of hallucination (Zhang et al., 2023). The *negative rejection* ability can be measured with the *rejection rate*, which was introduced by Chen et al. (2024). It is computed as the number of times the model answers with *NO-RES* when the documents indeed lack the required information, divided by the total number of such instances. High rejection rates indicate that the model effectively avoids generating potentially incorrect or misleading answers.

4 Results

In this section, we present the results for three types of models—base, instruct, and instruct + template—evaluated under various task instructions across different datasets, as detailed in the previous section.

4.1 Evaluation on Task Instruction I

In this initial set of experiments, we evaluate the models using Task Instruction I, pictured in Figure 1. Reported are the accuracies for each model/version combination at different levels of retrieved documents, that is, the number of documents added to the prompt of the LLM given the query. Unexpectedly, we find that the base models always outperform their instruct counterparts (with one exception), as evidenced by results in Table 1. Llama 2's base model outperforms its instruct counterpart (w/o template), averaged across retrieved documents, by 9.23 (48%) and 17.88 (+42%) points on NQ and Trivia QA, respectively. Similarly, Falcon's base model is 1.94 (+10%) and 7.48 (+20%) points better. Even more strongly, Llama 3's base model improves accuracy by 10.92 (+59%) and 37.5 (+186%); this is somewhat caused by Llama 3 reliance on its template; we'll examine this in Section 4.3. The only "half" exception is constituted by Mistral. In fact, Mistral's base model is -2.49 (-8%) less accurate than the instruct version on NQ. However, it is still 5.83 (+10%) more accurate on TriviaQA.

4.2 Evaluation on Task Instruction II

Intrigued by the first set of results, we proceed to examine a new task instruction developed to test the model's abilities to ground their answers. In this setting, models are required to provide a *Proof*, a piece of evidence to substantiate their answers based on the information present in the context documents. Examples of this setup are illustrated

Table 1: **Task Instruction I Accuracy** on NQ and TriviaQA. The abbreviations *C* and *I* denote the *Chat* and *Instruct* versions of the instruct models, respectively. The suffix *T* indicates instruct models using a *Template* to structure their responses. Accuracies are reporting at different levels of retrieved documents. With the, partial, exception Mistral, all base models outperform their instruct counterparts by a considerable margin.

	NQ # Retrieved Documents							**TriviaQA # Retrieved Documents**						
Model	1	2	3	4	5	8	10	1	2	3	4	5	8	10
Llama 2 7B	**23.88**	**24.71**	**27.83**	**29.53**	**30.22**	**31.01**	**31.46**	**55.85**	**57.15**	**59.28**	**60.40**	**61.24**	**62.93**	**63.89**
Llama 2 7B-C	16.06	18.48	18.62	18.59	19.21	21.98	21.08	32.59	34.63	41.09	42.95	46.14	49.19	48.96
Llama 2 7B-C + T	3.36	1.21	0.69	0.48	0.45	0.73	1.52	23.35	21.63	21.13	17.70	18.05	22.30	25.68
Llama 3 8B	**27.03**	**30.22**	**30.53**	**31.08**	**29.08**	**29.49**	**28.70**	**44.64**	**53.48**	**56.78**	**59.59**	**58.97**	**64.93**	**65.05**
Llama 3 8B-I	8.52	10.25	15.40	15.85	22.57	28.80	28.25	4.13	2.62	3.47	4.44	15.73	51.90	58.61
Llama 3 8B-I + T	14.40	17.34	19.04	15.30	18.10	20.35	19.35	16.45	27.94	31.32	30.80	34.18	44.04	50.12
Mistral 7B	**24.26**	**25.30**	25.72	26.17	27.66	26.65	28.87	**58.67**	**59.15**	**58.87**	**60.49**	**62.02**	**64.16**	**65.92**
Mistral 7B-I	20.04	24.99	**26.69**	**30.56**	**31.67**	**33.33**	**34.82**	48.85	52.31	54.72	55.90	56.97	59.06	60.60
Mistral 7B-I + T	18.17	23.54	19.14	27.90	27.45	27.07	26.79	45.88	50.22	52.42	53.73	54.37	57.56	58.31
Falcon 7B	**17.13**	**18.97**	**21.15**	**21.08**	**21.95**	**22.64**	**22.33**	**41.64**	**43.11**	**43.69**	**44.22**	**45.60**	**46.35**	**48.24**
Falcon 7B-I	15.68	17.72	17.96	19.21	20.08	20.04	20.98	33.19	36.52	36.46	37.71	38.42	38.83	39.33

Table 2: **Task Instruction II Accuracy** on NQ and TriviaQA, where a *Proof* is required. The abbreviations *C* and *I* denote the *Chat* and *Instruct* versions of the instruct models, respectively. The suffix *T* indicates instruct models using a *Template* to structure their responses. Accuracies are reporting at different levels of retrieved documents. In all cases considered, base models outperform their instruct counterparts by a considerable margin.

	NQ # Retrieved Documents							**TriviaQA # Retrieved Documents**						
Model	1	2	3	4	5	8	10	1	2	3	4	5	8	10
Llama 2 7B	**24.82**	**28.70**	**31.71**	**32.85**	**34.09**	**35.62**	**35.79**	**54.94**	**56.94**	**58.88**	**60.07**	**60.87**	**62.75**	**63.73**
Llama 2 7B-C	18.41	24.96	28.76	30.22	32.16	33.16	32.05	41.86	47.18	49.53	51.19	52.59	55.73	56.00
Llama 2 7B-C + T	1.59	2.42	3.88	5.75	8.45	12.81	12.91	4.22	6.55	8.75	10.15	14.90	20.69	27.48
Llama 3 8B	**29.39**	**31.53**	**34.72**	**37.07**	**36.59**	**39.15**	**40.22**	**61.57**	**63.32**	**64.56**	**65.93**	**66.52**	**67.67**	**68.04**
Llama 3 8B-I	18.83	23.57	25.65	27.97	30.84	34.13	37.97	42.67	44.44	47.11	48.10	48.71	52.59	56.48
Llama 3 8B-I + T	11.70	16.44	6.40	6.61	11.60	5.64	0.69	30.52	36.89	31.11	33.17	38.08	33.69	23.42
Mistral 7B	**24.82**	**30.74**	**33.75**	**35.17**	**36.83**	**40.15**	**40.15**	**57.98**	**60.49**	**62.18**	**63.70**	**65.23**	**67.04**	**67.44**
Mistral 7B-I	21.53	27.66	29.91	32.85	33.58	36.45	35.76	48.51	52.60	54.79	56.05	56.91	58.80	59.54
Mistral 7B-I + T	16.86	18.10	20.87	22.43	24.09	24.44	26.96	33.73	33.21	35.47	36.57	34.98	42.39	48.45
Falcon 7B	**17.58**	**20.04**	**22.43**	**23.75**	**23.85**	**26.12**	**26.72**	**40.32**	**42.76**	**44.92**	**45.45**	**46.47**	**47.23**	**48.57**
Falcon 7B-I	16.10	18.31	18.93	19.87	20.91	21.61	21.30	33.54	35.69	37.50	38.09	39.02	41.08	42.95

in Figure 4 and 5 for NQ and TriviaQA, respectively. Results can be seen in Table 2. We can immediately notice that there is a general upward shift in terms of accuracy for all models and settings; for instance, for Llama 2 base, it increases by 3.56 (+12%) on average. This probably indicates that asking for the proof is in itself a form of prompt engineering. Furthermore, we can observe that base models still outperform their counterparts. Actually, these results are even stronger, as in this setting Mistral base achieves higher accuracy than the instruct versions by 3.41 (+10%), too.

4.3 Instruct Models with Template

Our results highlight the difficulty that instruct models face in answering the question when the recommended template is used. Investigating this issue, we find that even though the instructions demand short answers, models in this setting override this specification and produce overly verbose responses, damaging their accuracy. This tendency may be linked to their fine-tuning and alignment for conversational purposes, where verbosity can be advantageous to assist users. This results in template-less instruct models exhibiting higher accuracy than their templated counterparts. For instance, in the NQ dataset using Task Instruction I, Llama 2's templated version barely achieves a 3% accuracy rate, while it only surpasses 10% when the context contains more than 8 documents under Task Instruction II. An additional notable aspect is

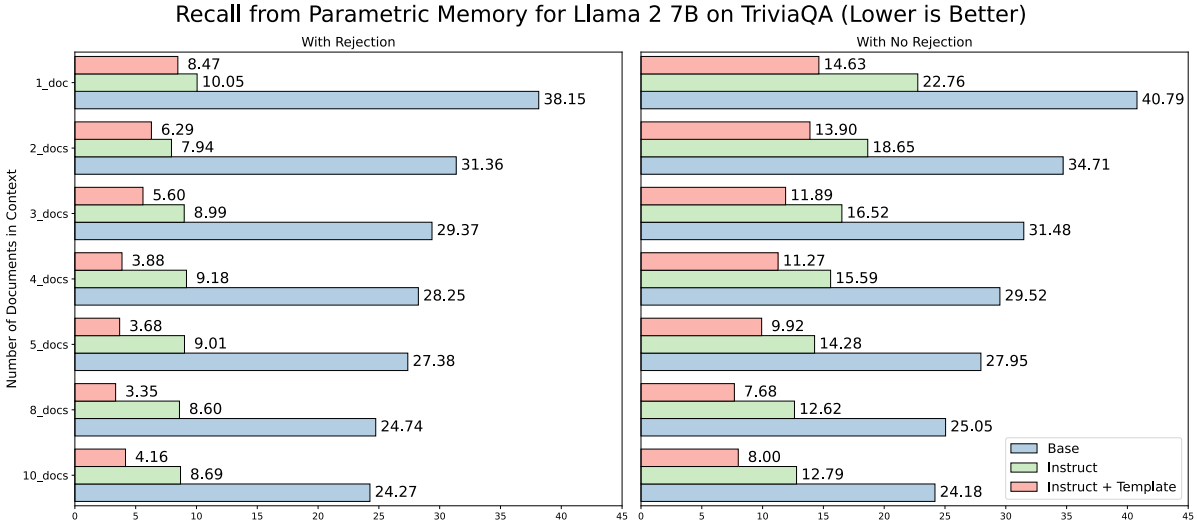

Figure 2: **Recalling from Parametric Memory - Llama 2 7B - TriviaQA**. Reported is the recall from parametric memory rate, defined as the number of instances where the model correctly answers despite the retrieved documents not containing the correct answer, divided by the number of times the answer is not present in the context. **(left)** Task Instruction I as shown in Figure 1; **(right)** No Rejection setting, where we do not specify to answer with *NO-RES* when the answer is not contained in the retrieved documents (example in Figure 6). In this case, the parametric memory rate increases for both model versions.

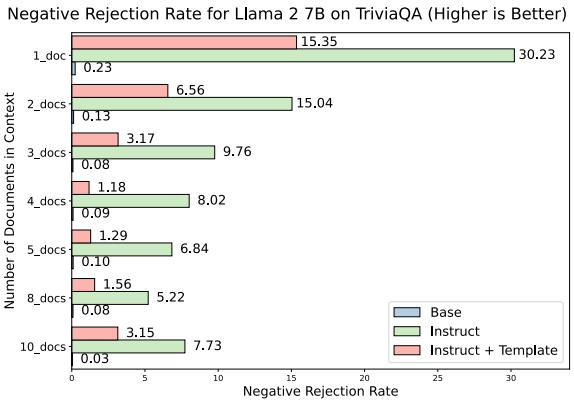

Figure 3: **Negative Rejection Rate - Llama 2 7B - TriviaQA.** Reported is the negative rejection rate, that is, the number of times the model answers *NO-RES* when the correct answer is not in the context, divided by the number of times the answer is indeed missing. Instruct models are much more effective at detecting such cases and following the instructions provided.

observed with Llama 3 instruct on Task Instruction I. When fewer than 4 documents are retrieved, the instruct model without a template fails to understand the prompt and generates random text. As the number of documents increases, however, its performance substantially improves, eventually surpassing the templated model. This suggests that a larger input length might have a role in overruling learned behavior.

5 Is Accuracy Sufficient?

Section 4 clearly indicates that base models outperform instruct models on RAG. But is that really the case? *Are base models truly better than the instruct counterpart on RAG-like prompts?* To answer this question, in this section, we go more in-depth in analyzing and comparing their behavior. First, we test the ability of these models to adhere to the task instructions. In particular, whether they appropriately respond with *NO-RES* when no relevant answer is present in the retrieved documents, which we call negative rejection rate.

5.1 Negative Rejection Rate

In Figure 3, we plot the negative rejection rates for various configurations (base, instruct, and instruct + template) of Llama 2 7B. It is evident that in most cases, the models fail to comply with the instruction to answer with *NO-RES* when the answer is absent. As an example, the instruct version of Llama 2 responds with *NO-RES* only 30.23% of the time when the answer is not in the one document context. The non-compliance to the task instruction is especially pronounced in the base model, which seldomly opts for rejection. It is worth noticing that as the number of documents in the context increases, all models tend to respond less frequently with *NO-RES*, suggesting both that a higher volume of documents might introduce more distract-

ing documents (Shi et al., 2023), leading the LLM to respond but erroneously, and that a larger input length might overrule task instructions.

5.2 Recall From Parametric Memory

Next, we consider cases where the correct answer is not present in the provided documents, yet the model still responds accurately. As illustrated in the left part of Figure 2, base models frequently manage to provide the correct answer even when there is none in the retrieved documents, suggesting that they "know" the answer from prior training. We call this "recall from parametric memory" (by parametric memory, we mean knowledge learned during training and stored in the parameters of the model, as opposed to non-parametric memory provided in the context through retrieved documents). Recall from parametric memory is not inherently problematic. A user might choose to both fine-tune on proprietary data and use RAG to get the highest possible accuracy. However, the specific instructions for this study emphasize that models should opt not to answer when the correct response is not evident in the documents. Not following this guideline raises important questions about their reliance on internal knowledge versus contextual information, particularly in settings where accurate rejection of unanswerable questions is crucial.

5.3 Evaluation with No Rejection

Here we aim to determine whether base models perform better simply because they do not adhere to these instructions, prompting us to continue our investigation by removing the requirement to respond with *NO-RES* in the prompt. As shown in the right part of Figure 2, instruct models also demonstrate a capacity to recall from parametric memory, although less frequently than their base counterparts. Moreover, evidence from Table 5 (appendix) suggests a slight improvement in accuracy for instruct models under these modified conditions. However, base models still outperform instruct ones. The results indicate that the processes of supervised fine-tuning and alignment detrimentally impact the model's capabilities in RAG. Moreover, a tradeoff is observed between the trustworthiness of the model and its ability to perform RAG effectively. As alignment and fine-tuning efforts enhance the reliability and adherence to desired behaviors, they simultaneously constrain the model's flexibility and efficiency in RAG tasks, highlighting a critical balance between trustworthiness and functional performance.

6 Related Works

Recent studies have highlighted challenges and potential improvements in language models' use of non-parametric versus parametric memory in question-answering tasks. Several papers (Krishna et al., 2021; Shi et al., 2024; Carlini et al., 2019; Kandpal et al., 2023; Mallen et al., 2023) demonstrate that LMs often rely on memorized answers, capable of responding correctly even when presented with irrelevant documents. Similarly, other studies (Longpre et al., 2021; Xie et al., 2024) observe that LMs continue to leverage their parametric knowledge despite prompt modifications with contrasting entities. Wu et al. (2024) describes this phenomenon as a balance between the model's inherent knowledge and its adherence to newly retrieved information, underscoring the ongoing challenge of enhancing model responsiveness to dynamic inputs. On enhancing reliance on provided content, Zhang et al. (2024) introduced a training strategy that emphasizes evidence-based responses, similar to the *Proof* mechanism in our QA tasks. This method has shown potential in improving model effectiveness by grounding responses in factual evidence, even though hallucination issues still remain an open problem (Zuccon et al., 2023; Gao et al., 2023). Cuconasu et al. (2024) found instruct models to be slightly more effective, but theirs was a controlled setting in which the ground truth was always provided. These insights collectively underline the intricate balance between leveraging learned knowledge and external data in improving QA systems, suggesting directions for future research in training strategies and model design.

7 Conclusions

In this paper, we aimed to systematically investigate the differences between LLM's base and instruct versions when used in RAG systems. Our findings reveal an unexpected outcome: base models exhibit superior performance on RAG tasks compared to their instructed and aligned counterparts. Further analysis indicates a tradeoff between accuracy and trustworthiness. This tradeoff calls for novel evaluation methodologies for RAG pipelines and suggests the necessity for mechanisms that afford users greater control in managing this tradeoff in a more direct and explicit manner.

8 Limitations

Our study is subject to several limitations. Primarily, due to computational resource constraints, we did not evaluate LLMs with more than 8B parameters, which might offer additional insights into the effectiveness of base versus instruct versions for RAG applications. Similarly, we quantize models to 4-bit. Additionally, our analysis could benefit from incorporating a broader range of datasets, particularly those that do not rely on Wikipedia as the primary knowledge source, or long-form QA datasets.

A critical limitation lies in the evaluation methodology. As noted in several studies (Katranidis and Barany, 2024; Yu et al., 2024), verifying the presence of the ground truth answer within the generated response can sometimes inaccurately penalize correct answers. This typically occurs if the ground truth is not fully captured in the response, even after normalization.

References

Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. 2023. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, Singapore. Association for Computational Linguistics.

Ebtesam Almazrouei, Hamza Alobeidli, Abdulaziz Alshamsi, Alessandro Cappelli, Ruxandra Cojocaru, Mérouane Debbah, Étienne Goffinet, Daniel Hesslow, Julien Launay, Quentin Malartic, Daniele Mazzotta, Badreddine Noune, Baptiste Pannier, and Guilherme Penedo. 2023. The falcon series of open language models. *Preprint*, arXiv:2311.16867.

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2023. Self-rag: Learning to retrieve, generate, and critique through self-reflection. *Preprint*, arXiv:2310.11511.

Nicholas Carlini, Chang Liu, Úlfar Erlingsson, Jernej Kos, and Dawn Song. 2019. The secret sharer: evaluating and testing unintended memorization in neural networks. In *Proceedings of the 28th USENIX Conference on Security Symposium*, SEC'19, page 267–284, USA. USENIX Association.

Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking large language models in retrieval-augmented generation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16):17754–17762.

Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for rag systems. *Preprint*, arXiv:2401.14887.

Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *Preprint*, arXiv:2401.08281.

DSPy. 2023. Dspy: Dynamic structured programming for python.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. 2023. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, Singapore. Association for Computational Linguistics.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-augmented generation for large language models: A survey. *Preprint*, arXiv:2312.10997.

Jiwoo Hong, Noah Lee, and James Thorne. 2024. Orpo: Monolithic preference optimization without reference model. *Preprint*, arXiv:2403.07691.

Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2023. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *Preprint*, arXiv:2311.05232.

Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised dense information retrieval with contrastive learning.

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. *Preprint*, arXiv:2007.01282.

Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Lélio Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7b. *Preprint*, arXiv:2310.06825.

Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547.

Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611, Vancouver, Canada. Association for Computational Linguistics.

Nikhil Kandpal, Haikang Deng, Adam Roberts, Eric Wallace, and Colin Raffel. 2023. Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23. JMLR.org.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, Online. Association for Computational Linguistics.

Vasileios Katranidis and Gabor Barany. 2024. Faaf: Facts as a function for the evaluation of generated text. *Preprint*, arXiv:2403.03888.

Kalpesh Krishna, Aurko Roy, and Mohit Iyyer. 2021. Hurdles to progress in long-form question answering. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4940–4957, Online. Association for Computational Linguistics.

Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti,

Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466.

LangChain. 2023. Langchain: Building applications with llms through composability.

Kenton Lee, Ming-Wei Chang, and Kristina Toutanova. 2019. Latent retrieval for weakly supervised open domain question answering. In *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pages 6086–6096, Florence. Association for Computational Linguistics.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *Preprint*, arXiv:2307.03172.

Shayne Longpre, Kartik Perisetla, Anthony Chen, Nikhil Ramesh, Chris DuBois, and Sameer Singh. 2021. Entity-based knowledge conflicts in question answering. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 7052–7063, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.

Sewon Min, Danqi Chen, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2019. A discrete hard EM approach for weakly supervised question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2851–2864, Hong Kong, China. Association for Computational Linguistics.

Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. 2022. Training language models to follow instructions with human feedback. *Preprint*, arXiv:2203.02155.

Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Hamza Alobeidli, Alessandro Cappelli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The refinedweb dataset for falcon llm: Outperforming curated corpora with web data only. In *Advances in Neural Information Processing Systems*, volume 36, pages 79155–79172. Curran Associates, Inc.

Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, and Chelsea Finn. 2023. Direct preference optimization: Your language model is secretly a reward model. *Preprint*, arXiv:2305.18290.

Md Masudur Rahman and Yexiang Xue. 2022. Robust policy optimization in deep reinforcement learning. *Preprint*, arXiv:2212.07536.

Noam Shazeer. 2019. Fast transformer decoding: One write-head is all you need. *Preprint*, arXiv:1911.02150.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. *Preprint*, arXiv:2302.00093.

Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. 2024. Detecting pretraining data from large language models. *Preprint*, arXiv:2310.16789.

Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval augmentation reduces hallucination in conversation. *arXiv preprint arXiv:2104.07567*.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B Hashimoto. 2023. Alpaca: A strong, replicable instruction-following model. *Stanford Center for Research on Foundation Models. https://crfm. stanford. edu/2023/03/13/alpaca. html*, 3(6):7.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, Dan Bikel, Lukas Blecher, Cristian Canton Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura,

Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Ranjan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurelien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. 2023. Llama 2: Open foundation and fine-tuned chat models. *Preprint*, arXiv:2307.09288.

Hongru Wang, Wenyu Huang, Yang Deng, Rui Wang, Zezhong Wang, Yufei Wang, Fei Mi, Jeff Z Pan, and Kam-Fai Wong. 2024. Unims-rag: A unified multi-source retrieval-augmented generation for personalized dialogue systems. *arXiv preprint arXiv:2401.13256*.

Xuezhi Wang, Haohan Wang, and Diyi Yang. 2022. Measure and improve robustness in nlp models: A survey. *Preprint*, arXiv:2112.08313.

Kevin Wu, Eric Wu, and James Zou. 2024. Clasheval: Quantifying the tug-of-war between an llm's internal prior and external evidence. *Preprint*, arXiv:2404.10198.

Jian Xie, Kai Zhang, Jiangjie Chen, Renze Lou, and Yu Su. 2024. Adaptive chameleon or stubborn sloth: Revealing the behavior of large language models in knowledge conflicts. In *The Twelfth International Conference on Learning Representations*.

Shusheng Xu, Wei Fu, Jiaxuan Gao, Wenjie Ye, Weilin Liu, Zhiyu Mei, Guangju Wang, Chao Yu, and Yi Wu. 2024. Is dpo superior to ppo for llm alignment? a comprehensive study. *Preprint*, arXiv:2404.10719.

Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2024. Evaluation of retrieval-augmented generation: A survey. *Preprint*, arXiv:2405.07437.

Tianjun Zhang, Shishir G. Patil, Naman Jain, Sheng Shen, Matei Zaharia, Ion Stoica, and Joseph E. Gonzalez. 2024. Raft: Adapting language model to domain specific rag. *Preprint*, arXiv:2403.10131.

Yue Zhang, Yafu Li, Leyang Cui, Deng Cai, Lemao Liu, Tingchen Fu, Xinting Huang, Enbo Zhao, Yu Zhang, Yulong Chen, Longyue Wang, Anh Tuan Luu, Wei Bi, Freda Shi, and Shuming Shi. 2023. Siren's song in the ai ocean: A survey on hallucination in large language models. *Preprint*, arXiv:2309.01219.

Yichen Zhu, Zhicai Ou, Xiaofeng Mou, and Jian Tang. 2024. Retrieval-augmented embodied agents. *Preprint*, arXiv:2404.11699.

Guido Zuccon, Bevan Koopman, and Razia Shaik. 2023. Chatgpt hallucinates when attributing answers. In *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, SIGIR-AP '23, page 46–51, New York, NY, USA. Association for Computing Machinery.

A More Details on Datasets and Models

A.1 Dataset

We employ the NQ-open and TriviaQA-unfiltered datasets for our evaluations. For NQ-open we adopt the processing procedure of Cuconasu et al. (2024), resulting in 2,889 test examples. For TriviaQA-unfiltered, we adhere to the validation and test split adopted in previous studies (Min et al., 2019; Asai et al., 2023), using 11,313 test queries for evaluation. The two datasets use the English Wikipedia dated 20 December 2018 as a knowledge source. Following the Dense Passage Retrieval (DPR) approach (Karpukhin et al., 2020), we split each Wikipedia article into non-overlapping passages of 100 words.

Both datasets feature questions that allow for multiple valid answers. These can range from synonymous terms, such as "New York" and "NY", to questions accepting multiple distinct correct answers. TriviaQA is notable for its diversity in acceptable responses, with some queries accommodating up to 400 valid answers. This variance significantly lowers the likelihood of a correct response being marked as incorrect purely based on phrasing variances, contributing to notably higher accuracy scores on TriviaQA than on NQ-open, as discussed in Section 4.

A.2 Retriever

The retriever used to select the top-k documents is Contriever (Izacard et al., 2021), which is a BERT-based dense model trained using unsupervised contrastive loss. The embedding of each document and query is obtained by averaging the hidden state of the last layer of the model. For document retrieval from the corpus, we employ a FAISS index (Douze et al., 2024; Johnson et al., 2019) by using an inner product similarity metric (IndexFlatIP) with an exhaustive search.

To assess the availability of an answer within the provided documents, we compute the top-k accuracy of the retriever. This metric evaluates how often the ground truth answer appears within the top-k documents retrieved for a query. Scores can be seen in Table 3.

Table 3: **Retriever top-k accuracy.**

Contriever	**# Retrieved Documents**						
Dataset	1	2	3	4	5	8	10
NQ	25.02	35.69	42.89	47.84	51.33	57.84	60.85
TriviaQA	39.15	50.70	56.45	60.32	63.03	68.35	70.49

Table 4: **Closed-book Accuracy** of the models. The task instruction used in this case for both datasets can be seen in Figure 7.

Model	**NQ Closed-Book**	**TriviaQA Closed-Book**
Llama 2 7B	**25.20**	**55.57**
Llama 2 7B-C	16.51	37.33
Llama 2 7B-C + T	13.12	41.73
Llama 3 8B	22.01	**57.37**
Llama 3 8B-I	27.17	25.46
Llama 3 8B-I + T	**27.80**	54.87
Mistral 7B	**28.11**	**60.68**
Mistral 7B-I	18.76	47.18
Mistral 7B-I + T	15.96	49.30
Falcon 7B	**15.26**	**41.12**
Falcon 7B-I	12.74	27.83

A.3 Generative Models

We utilize publicly available, open-weight LLMs accessible via Hugging Face. All models are quantized to 4-bit using the bitsandbytes library[3] to optimize computational efficiency. We perform all experiments with a single Nvidia RTX4090 GPU.

Here is a brief description of the main characteristics of each model:

Llama 2 7B. The 7B parameters versions[4] of the Llama 2 family (Touvron et al., 2023) are pretrained on publicly available data and optimized for a range of natural language generation tasks. This series features a context length of 4096 tokens, and the 7B version employs multi-query attention (MQA) (Shazeer, 2019) to enhance processing efficiency and response quality.

Llama 3 8B. The Llama 3 series[5] builds on the architecture and improvements of its predecessors, offering models with 8B parameters[6]. It employs group-query attention (GQA) (Ainslie et al., 2023) and extends the context length to 8192 tokens, thus facilitating enhanced language generation across a broad range of tasks.

[3] https://huggingface.co/docs/bitsandbytes/main/en/index

[4] https://huggingface.co/meta-llama/Llama-2-7b-hf https://huggingface.co/meta-llama/Llama-2-7b-chat-hf

[5] Llama 3 Blog Page

[6] https://huggingface.co/meta-llama/Meta-Llama-3-8B https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct

Table 5: **Task Instruction I with No Rejection Percentage Accuracy** on NQ and TriviQA. In the No Rejection setting, we do not specify to answer with *NO-RES* when the answer is not contained in the retrieved documents. An example of the task instruction adopted in these experiments can be seen in Figure 6.

	NQ # Retrieved Documents							**TriviaQA # Retrieved Documents**						
Model	1	2	3	4	5	8	10	1	2	3	4	5	8	10
Llama 2 7B	**26.41**	**28.11**	**30.49**	**31.15**	**31.57**	**32.02**	**31.81**	**58.09**	**59.98**	**61.07**	**61.85**	**62.22**	**63.44**	**64.26**
Llama 2 7B-C	19.76	25.20	26.00	27.62	27.35	27.76	28.31	47.29	50.61	52.93	54.27	55.06	57.11	57.71
Llama 2 7B-C + T	4.05	2.53	1.56	1.25	1.11	1.90	3.63	33.46	39.70	41.32	41.34	41.09	39.01	43.11
Llama 3 8B	**28.14**	**30.32**	**30.46**	**30.81**	28.66	29.21	28.94	**49.65**	**57.45**	**61.07**	**63.17**	**64.72**	**66.53**	**66.80**
Llama 3 8B-I	13.36	19.21	23.19	25.75	**29.53**	**31.39**	**31.15**	4.05	3.94	5.23	7.01	18.41	53.94	61.46
Llama 3 8B-I + T	14.88	13.33	12.81	15.40	20.32	21.98	21.98	26.85	37.59	42.25	44.53	46.36	53.15	56.00
Mistral 7B	**25.48**	**26.48**	26.13	26.83	29.39	29.15	29.49	**59.52**	**60.06**	**60.90**	**62.17**	**64.65**	**66.45**	**67.88**
Mistral 7B-I	21.53	26.31	**29.42**	**31.36**	**32.09**	**34.06**	**34.89**	51.87	54.42	56.19	57.07	58.07	59.93	61.25
Mistral 7B-I+ T	19.56	24.96	26.76	28.73	27.83	28.04	36.55	48.15	51.99	54.48	55.36	56.34	58.86	59.85
Falcon 7B	**16.58**	**19.45**	**19.80**	**21.46**	**21.08**	**21.08**	**21.26**	**41.66**	**43.55**	**43.40**	**43.92**	**45.04**	**45.62**	**45.54**
Falcon 7B-I	16.03	18.28	18.90	19.38	19.42	19.38	19.97	33.99	36.08	36.78	36.98	37.89	38.18	38.43

Table 6: **Task Instruction II Answer Coherence Percentage Accuracy** on NQ and TriviaQA. It indicates the number of times the generated answer is contained in the generated *Proof*, over the total number of instances.

	NQ # Retrieved Documents							**TriviaQA # Retrieved Documents**						
Model	1	2	3	4	5	8	10	1	2	3	4	5	8	10
Llama 2 7B	36.93	41.16	41.64	41.43	42.16	**42.99**	**41.26**	37.22	44.27	44.68	47.19	48.00	**50.35**	**47.14**
Llama 2 7B-C	**45.07**	**47.35**	**46.97**	**46.45**	**45.17**	40.53	32.78	**43.15**	**49.61**	**53.09**	**53.46**	**48.92**	40.14	28.93
Llama 2 7B-C + T	0	0.59	1.14	0.07	0.07	0	0	0.05	0.46	0.75	0.11	0.04	0	0
Llama 3 8B	36.90	41.36	43.54	**46.49**	46.80	**51.78**	51.37	**51.66**	**57.44**	**59.46**	**61.30**	**62.02**	**64.12**	**63.58**
Llama 3 8B-I	**42.02**	**51.92**	**50.71**	43.86	**48.36**	48.29	**53.10**	35.45	50.55	50.89	48.91	55.02	55.28	56.78
Llama 3 8B-I + T	0.52	0.03	0.10	0.14	1.38	0.28	0.07	1.10	0.13	0.12	0.49	1.59	1.42	0.5
Mistral 7B	36.73	44.03	46.56	45.66	51.47	54.62	53.31	43.23	52.16	**56.07**	**58.48**	**60.82**	**66.66**	**68.04**
Mistral 7B-I	**50.12**	**53.24**	**54.38**	**55.49**	**57.39**	**57.74**	**56.07**	**51.08**	**53.52**	55.43	57.53	58.55	59.68	58.58
Mistral 7B-I+ T	15.99	10.90	7.65	5.75	3.53	4.12	2.70	16.82	14.87	11.42	7.90	5.70	2.45	2.16
Falcon 7B	1.87	2.08	3.32	3.01	3.81	4.50	1.15	3.02	3.70	3.47	4.52	4.86	3.17	1.61
Falcon 7B-I	**3.39**	**3.12**	**3.77**	**4.85**	**4.26**	**1.73**	**1.04**	**4.45**	**5.94**	**6.53**	**6.08**	**6.28**	**3.34**	**1.67**

Mistral 7B. Developed as a highly efficient model with 7B parameters[7], Mistral (Jiang et al., 2023) focuses on delivering high performance and accuracy in text generation. It uses GQA and sliding window attention with an 8192-token context length.

Falcon 7B. The Falcon 7B[8] is the smallest model of the Falcon series (Almazrouei et al., 2023) and was trained on the RefinedWeb dataset (Penedo et al., 2023)—a large, filtered, and deduplicated corpus. Similarly to Llama2 7B, it uses MQA, but with a smaller context length of 2048 tokens. Unlike the other models, the instruct version of Falcon 7B was not specifically trained using a fixed template, which is why no separate "instruct + template" variant is listed in any figure or table.

The closed-book accuracy of the models is detailed in Table 4. In this scenario, models are evaluated without any documents in their prompt, necessitating a modification to Task Instruction I. An example of this modified task instruction can be viewed in Figure 7.

B Further Analysis with Task Instruction II

In this section, we examine more in detail whether models can justify their answers with *Proof*. We specifically investigate whether even the base models can adhere to instructions and provide accurate *Proof* for their responses.

Table 6 shows the percentage of instances where

[7]hhttps://huggingface.co/mistralai/Mistral-7B-v0.1
https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.1

[8]https://huggingface.co/tiiuae/falcon-7b
https://huggingface.co/tiiuae/falcon-7b-instruct

the generated answer, whether correct or incorrect, is included in the generated *Proof*, which we will refer to as "coherence". The inclusion of the answer in the *Proof* indicates that the model's responses align with the information provided in the documents. This measure, however, does not necessarily reflect answer correctness, as it only assesses coherence with the documented evidence.

As observed in Table 6, base models are "coherent" with their answers, often outperforming their instruct counterparts. Mistral notably achieves the highest coherence score, reaching 68% with 10 retrieved documents, while Falcon exhibits the lowest, often failing to provide any *Proof* at all. Even the instruct version of Falcon typically offers only the direct answer without supporting evidence.

However, as discussed in the limitations section (Section 8), the presence of an answer in the *Proof* does not guarantee its "coherence" accuracy. An answer may be included, but the *Proof* might not actually derive from the provided documents. Additionally, even if the answer is present in the *Proof*, it may not be recognized as valid due to the inclusion of additional text in the response. For example, if a model begins its response with "The answer to the question is..." and then reports an answer that is technically part of the *Proof*, this response might still be deemed invalid because the introductory phrase does not originate from the context documents.

C Further Analysis on the Negative Rejection Rates

In Section 5.1, we discussed the negative rejection ability of Llama 2 on TriviaQA. This section extends the discussion to the behavior of other models on both datasets, NQ and TriviaQA. From Tables 8 and 9, it is evident that the instruct models generally exhibit higher negative rejection rates compared to their base counterparts. However, each model follows its own trend.

Llama 2 and Mistral demonstrate similar behaviors in that, as the number of documents in the context increases, their tendency to answer with *NO-RES* decreases. Notably, the instruct version with a template for Llama 2 rarely rejects to respond when the answer is not present in the context documents. In contrast, the Mistral instruct version with a template shows a significantly stronger negative rejection ability than without the template, reaching up to 42.98% score with one retrieved document on NQ.

Llama 3 exhibits a distinct trend, maintaining a mean negative rejection rate of 34% with the template, and 35.57% without. However, when the model is not using the template, similar to Llama 2 and Mistral, the rejection rates for Llama 3 decline with an increase in document count. Indeed, it shows a significant reduction of 31.7 (-70%) passing from 45.16% to 13.46% when the number of retrieved documents increases from 5 to 8 on NQ; while for TriviaQA the rate drops by 14.86 (-50%) in the same situation.

Falcon models show the least tendency to respond with *NO-RES*, particularly in the instruct and instruct-template settings, where rejection rates are consistently low or even non-existent in some configurations. This behavior indicates a propensity to generate answers even when the information is not present, potentially leading to higher rates of hallucination. The primary reason for this is Falcon's difficulty in following instructions effectively.

<table>
<tr><th>Mistral 7B</th><th>Mistral 7B-Instruct + Template</th></tr>
<tr><td>You are given a question and you MUST respond by EXTRACTING the answer (max 5 tokens) from one of the provided documents. If none of the documents contain the answer, respond with NO-RES.
START example
Document [209707](Title: Ancient Egyptian technology) Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty. Chariots, however, are only believed to have been introduced by the invasion of the Hyksos in the Second Intermediate period; during the New Kingdom era, chariotry became central to Egypt's military.
Question: when was the potter's wheel first used in egypt
Answer: 4th Dynasty
Proof: Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty.
END example</td><td>[INST] You are given a question and you MUST respond by EXTRACTING the answer (max 5 tokens) from one of the provided documents. If none of the documents contain the answer, respond with NO-RES.
START example
Document [209707](Title: Ancient Egyptian technology) Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty. Chariots, however, are only believed to have been introduced by the invasion of the Hyksos in the Second Intermediate period; during the New Kingdom era, chariotry became central to Egypt's military.
Question: when was the potter's wheel first used in egypt
Answer: 4th Dynasty
Proof: Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty.
END example</td></tr>
<tr><td colspan="2">Documents:
Document [12919694](Title: User State Migration Tool) User State Migration Tool The User State Migration Tool (USMT) is a Microsoft command line utility program intended to allow advanced users, comfortable with Scripting languages, to transfer files and settings between PCs. This task is also performed by Windows Easy Transfer, recommended for general users...
Document [12919696](Title: User State Migration Tool) Settings Transfer Wizard and Windows Easy Transfer. USMT is designed to perform mass migrations and allows customizations designed for corporate environments...
Document [12919695](Title: User State Migration Tool) there have been several attempts to provide access to its useful functionality by creating GUI wrappers for it...

Question: what tool is available from microsoft as an alternative to the usmt for smaller migrations</td></tr>
<tr><td>Answer: Windows Easy Transfer
Proof: User State Migration Tool The User State Migration Tool (USMT) is a Microsoft command line utility program intended to allow advanced users, comfortable with Scripting languages, to transfer files and settings between PCs. This task is also performed by Windows Easy Transfer, recommended for general users...</td><td>Answer: [/INST] NO-RES.</td></tr>
</table>

Figure 4: **Base vs. Instruct + Template under Task Instruction II on NQ**. This comparison of responses between the base and instruct + template versions of Mistral 7B illustrates an example where the base model correctly identifies the answer, while the instruct + template version erroneously opts for a *NO-RES* response, despite the correct answer being present in the documents. *Italic* text denotes the template.

<table>
<tr><th>Llama 2 7B</th><th>Llama 2 7B-Chat + Template</th></tr>
<tr><td>You are given a question and you MUST respond by EXTRACTING the answer from one of the provided documents. If none of the documents contain the answer, respond with NO-RES.
START example
Document [209707](Title: Ancient Egyptian technology) Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty. Chariots, however, are only believed to have been introduced by the invasion of the Hyksos in the Second Intermediate period; during the New Kingdom era, chariotry became central to Egypt's military.
Question: when was the potter's wheel first used in egypt
Answer: 4th Dynasty
Proof: Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty.
END example</td><td>[INST] «SYS»
You are given a question and you MUST respond by EXTRACTING the answer from one of the provided documents. If none of the documents contain the answer, respond with NO-RES.
START example
Document [209707](Title: Ancient Egyptian technology) Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty. Chariots, however, are only believed to have been introduced by the invasion of the Hyksos in the Second Intermediate period; during the New Kingdom era, chariotry became central to Egypt's military.
Question: when was the potter's wheel first used in egypt
Answer: 4th Dynasty
Proof: Evidence indicates that Egyptians made use of potter's wheels in the manufacturing of pottery from as early as the 4th Dynasty.
END example
«/SYS»</td></tr>
<tr><td colspan="2">Documents:
Document [1](Title: Batman Returns) the Penguin. We didn't really officially cast it, but for a short nasty little guy, it's a short list. I ended up writing the character for Danny DeVito. Burgess Meredith (who portrayed the Penguin in the 1960s TV series "Batman") was cast for a little cameo as Tucker Cobblepot...
Document [2](Title: Batman: Mystery of the Batwoman) This is the only time in the DC animated universe that Paul Williams did not voice the Penguin...
Document [3](Title: The Penguin's a Jinx) The Penguin goes to Wayne Manor and returns the actress. He then uses his gas-umbrella to knock out anyone inside the statues...

Question: Who played the part of 'The Penguin' in the TV series 'Batman'?</td></tr>
<tr><td>Answer: Burgess Meredith
Proof: Document [1](Title: Batman Returns) the Penguin. We didn't really officially cast it, but for a short nasty little guy, it's a short list. I ended up writing the character for Danny DeVito. Burgess Meredith (who portrayed the Penguin in the 1960s TV series "Batman") was cast for a little cameo as Tucker Cobblepot...</td><td>Answer: [/INST] Danny DeVito
Proof: Document [1](Title: Batman Returns) the Penguin. We didn't really officially cast it, but for a short nasty little guy, it's a short list. I ended up writing the character for Danny DeVito.</td></tr>
</table>

Figure 5: **Base vs. Instruct + Template under Task Instruction II on TriviaQA**. This comparison of responses between the base and instruct + template versions of Llama 2 7B illustrates an example where the base model correctly identifies the answer, while the instruct + template version inaccurately attributes the answer to a different actor. Nevertheless, in both cases, the answers are "coherent" with the *Proof* since each evidence contains the generated answer. *Italic* text denotes the template.

<table>
<tr><th>Falcon 7B</th><th>Falcon 7B-Instruct</th></tr>
<tr><td>You are given a question and you MUST respond by EXTRACTING the answer (max 5 tokens) from one of the provided documents.</td><td>You are given a question and you MUST respond by EXTRACTING the answer (max 5 tokens) from one of the provided documents.</td></tr>
<tr><td colspan="2">Documents:
Document [1590238](Title: 1970s in film) Thanks to a stellar cast, experienced director, and a poignant story, "Network" became one of the largest critical successes of 1976. Another film, Rocky, about a clubhouse boxer (played by Sylvester Stallone) who is granted a world championship title fight won the Best Picture Academy Award that year. The film also became a major commercial success and spawned four sequels through the rest of the 1970s and 1980s...
Document [2832642](Title: Ned Beatty) 1975. Beatty received his first Academy Award nomination for Best Supporting Actor category for the acclaimed film "Network" (1976), portraying a television network's bombastic but shrewd chairman of the board who convinces the mad Howard Beale character (portrayed by Peter Finch) that corporation-led global dehumanization is not only inevitable, but is also a good thing...

Question: who won the oscar for best picture in 1976</td></tr>
<tr><td>Answer: Rocky</td><td>Answer: "Network"</td></tr>
</table>

Figure 6: **Base vs. Instruct under Task Instruction I with No Rejection on NQ**. This figure presents responses under a No Rejection setting, where models are not tasked with responding with *NO-RES* if the answer is not contained in the retrieved documents. It compares the base and instruct versions of Falcon 7B. In this instance, the base model accurately identifies "Rocky" as the Oscar winner for Best Picture in 1976, while the instruct version incorrectly cites "Network".

<table>
<tr><th>Llama 3 8B</th><th>Llama 3 8B-Instruct + Template</th></tr>
<tr><td>You are given a question and you MUST respond with a short answer based on your internal knowledge. If you do not know the answer, please respond with NO-RES.</td><td><|<i>start_header_id</i>|><i>system</i><|<i>end_header_id</i>|>

You are given a question and you MUST respond with a short answer based on your internal knowledge. If you do not know the answer, please respond with NO-RES.<|<i>eot_id</i>|>
<|<i>start_header_id</i>|><i>user</i><|<i>end_header_id</i>|></td></tr>
<tr><td colspan="2">Question: In which US city did the 2004 remake of the film Alfie take place?</td></tr>
<tr><td><b>Answer:</b> New York City</td><td><b>Answer:</b> <|<i>eot_id</i>|><|<i>start_header_id</i>|><i>assistant</i>
<|<i>end_header_id</i>|> London!</td></tr>
</table>

Figure 7: **Base vs. Instruct + Template under Closed-Book QA on TriviaQA**. This figure compares responses from the base and instruct + template versions of Llama 3 8B for a question in a closed-book setting, where no additional documents are provided. The example demonstrates how the base model accurately identifies "New York City" as the setting of the 2004 remake of the film Alfie, whereas the instruct + template version erroneously claims the location as "London". *Italic* text denotes the template.

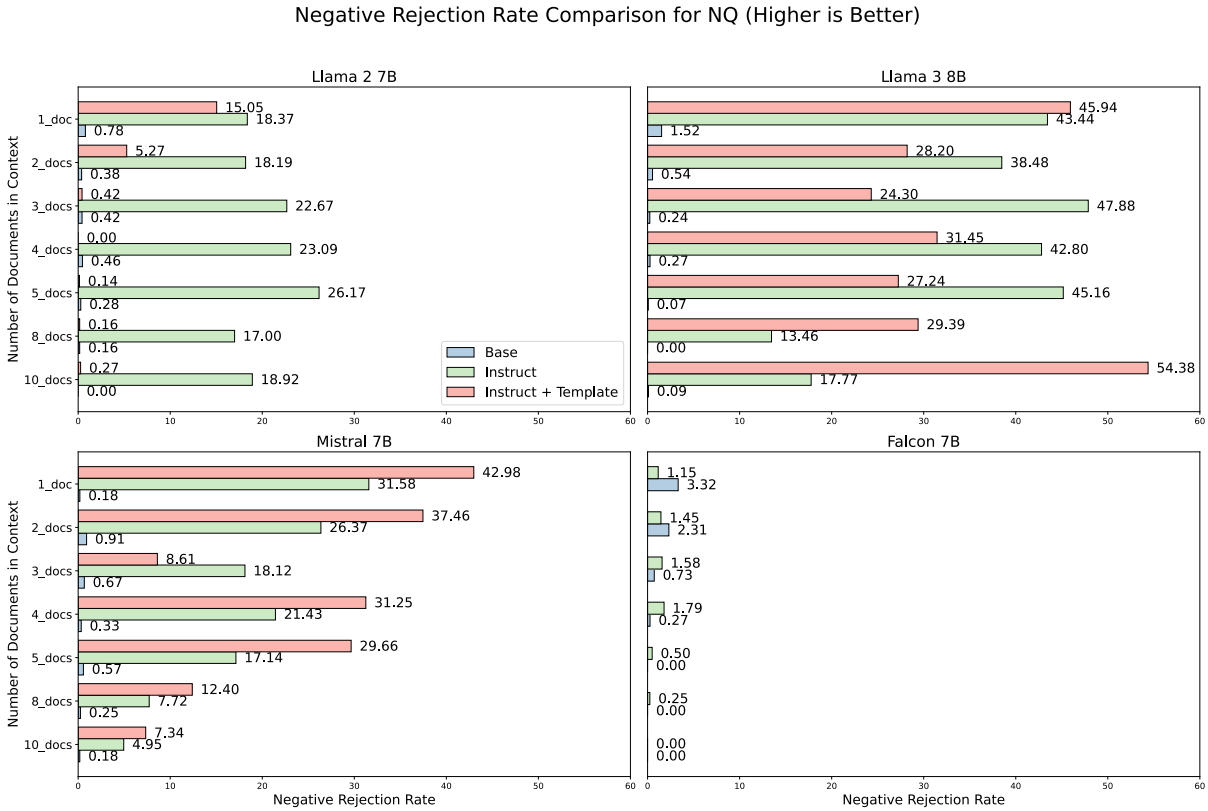

Figure 8: **Negative Rejection Comparison for NQ.** Reported is the negative rejection rate, that is, the number of times the model answers *NO-RES* when the correct answer is not in the context, divided by the number of times the answer is indeed missing. Instruct models are much more effective at detecting such cases and following the instructions provided.

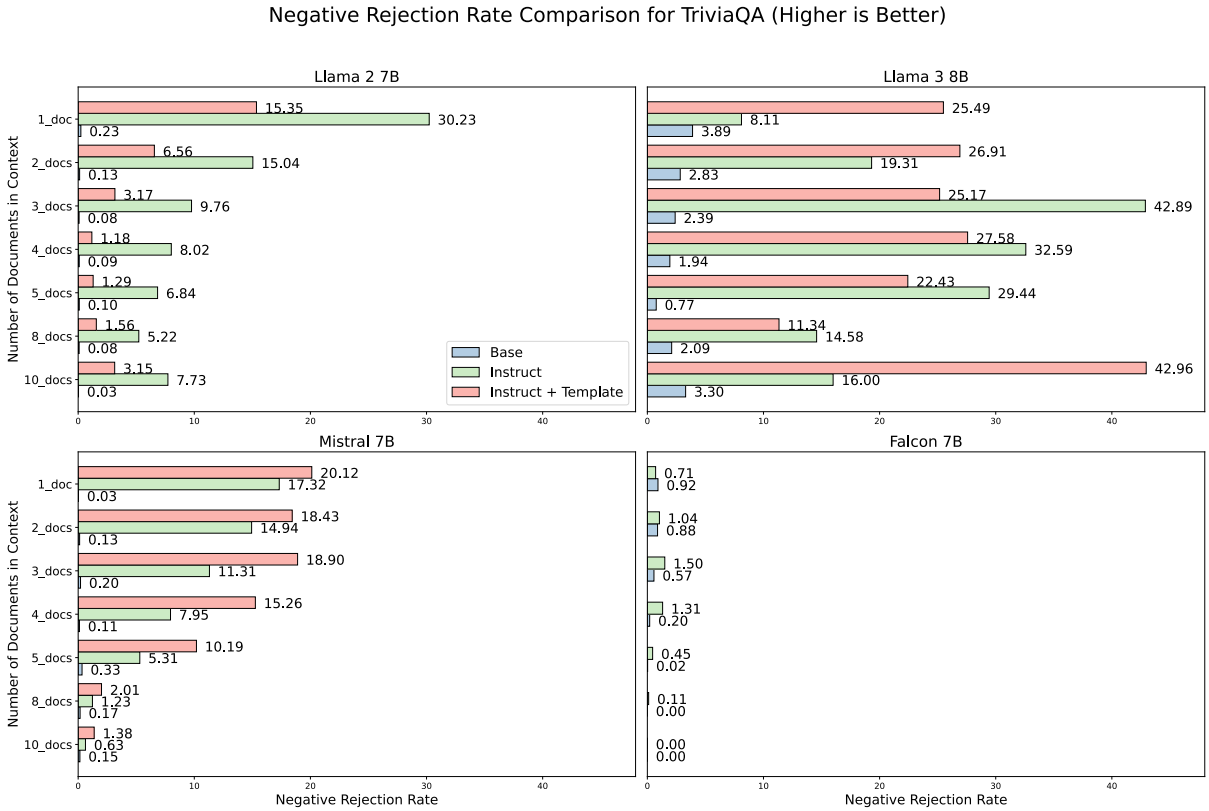

Figure 9: **Negative Rejection Comparison for TriviaQA.** Reported is the negative rejection rate, that is, the number of times the model answers *NO-RES* when the correct answer is not in the context, divided by the number of times the answer is indeed missing. Instruct models are much more effective at detecting such cases and following the instructions provided.